

|                                                                                                                    |                                                                |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Uniwersytet Radomski im. Kazimierza Pułaskiego w Radomiu                                                           |                                                                |
| Wydział Transportu, Elektrotechniki i Informatyki                                                                  |                                                                |
| Techniczne zastosowania sieci neuronowych                                                                          |                                                                |
| Rok akademicki 2025/2026                                                                                           |                                                                |
| <b>Projekt: Zastosowanie sieci samouczącej się Kohena do przetworzenia obrazów i tworzenia efektów graficznych</b> |                                                                |
| Autorzy:                                                                                                           | Ivan Pohodaev<br>Andrii<br>Yevlakh<br>Maksymilian<br>Wieczorek |

## Cel

Zastosowanie sieci samouczącej się Kohena do przetworzenia obrazów i tworzenia efektów graficznych

### 1. Struktura sieci neuronowej

W projekcie zaimplementowano samoorganizującą się mapę cech (SOM), czyli sieć typu Kohonena opartą na uczeniu nienadzorowanym. Sieć posiada dwa wejścia odpowiadające współrzędnym  $x$  oraz  $y$  punktów w przestrzeni dwuwymiarowej. Topologia warstwy wyjściowej jest konfigurowalna i może przybrać formę prostokątnej siatki o wymiarach  $h \times w$  (domyślnie 100 neuronów) lub jednowymiarowego łańcucha, który może być otwarty lub zamknięty w pętlę. Każdy neuron w strukturze posiada wektor wag  $\mathbf{w} = [w_x, w_y]$ , inicjowany losowo w przestrzeni jednostkowej. Zamiast klasycznych funkcji aktywacji, sieć wykorzystuje odległość euklidesową do wyłonienia neuronu zwycięzcy (BMU), a połączenia między neuronami w warstwie wyjściowej definiują relację sąsiedztwa, która determinuje zakres aktualizacji wag podczas procesu uczenia.[1,2]

### 2. Opis algorytmu uczenia

Proces adaptacji sieci opiera się na klasycznym algorytmie uczenia nienadzorowanego wykorzystującym regułę Kohonena, która realizuje samoorganizację mapy poprzez iteracyjną modyfikację wektorów wag neuronów. Procedura rozpoczyna się od losowej inicjalizacji wag  $\mathbf{W}$  w przestrzeni cech, po czym w każdym kroku algorytmu z zestawu danych losowany jest punkt wejściowy  $p$ . Na jego podstawie wyznaczany jest neuron zwycięzca, określany mianem BMU (*Best Matching Unit*), co odbywa się poprzez minimalizację odległości euklidesowej między wektorem wejściowym a wagami poszczególnych jednostek w strukturze. Po zidentyfikowaniu BMU następuje aktualizacja wag zwycięzcy oraz neuronów znajdujących się w jego sąsiedztwie topologicznym, zdefiniowanym przez promień  $S$ . Korekta wartości przeprowadzana jest zgodnie z regułą:

$$\mathbf{w}_i(t+1) = \mathbf{w}(t) + \eta(t) * [p(t) - \mathbf{w}_i(t)]$$

**Wzór: Reguła aktualizacji wag w uczeniu konkurencyjnym**

- $\mathbf{W}_i$  - wektor wag  $i$ -tego neuronu
- $t$  - aktualny krok iteracji (czas)
- $\eta$  - współczynnik szybkości uczenia
- $P$  - wektor wejściowy (punkt danych)

gdzie  $\eta$  oznacza współczynnik szybkości uczenia. Charakterystyczną cechą zaimplementowanego procesu jest stopniowe wygaszanie parametrów  $\eta$  oraz  $S$  przy użyciu współczynnika 0.9998 po każdej iteracji, co pozwala na przejście od globalnej reorganizacji struktury do precyzyjnego dopasowania lokalnego. Algorytm wzbogacono o mechanizm dynamicznego przywracania wyższych wartości parametrów w momencie zmiany docelowego kształtu danych, co zapewnia elastyczność sieci i umożliwia płynne morfowanie mapy pomiędzy różnymi rozkładami punktów. [1,2]

### 3. Ciąg uczący (CU)

Zbiór danych wykorzystywany w procesie samoorganizacji mapy składa się z 2000 elementów generowanych dynamicznie przy użyciu dedykowanych procedur matematycznych. Każdy rekord ciągu stanowi dwuwymiarowy wektor reprezentujący współrzędne punktu  $[x, y]$  w przestrzeni jednostkowej, co umożliwia formowanie zróżnicowanych kształtów geometrycznych, takich jak koło, kwadrat, trójkąt, romb, trapez oraz gwiazda. Proces generowania danych opiera się na losowaniu wartości z rozkładów jednostajnych, które są następnie poddawane transformacjom, aby uzyskać pożądany rozkład punktów wewnątrz zadanych granic obszaru. Przykładowe wektory dla zbioru reprezentującego koło mogą przyjmować postać  $[0.52, 0.48]$ ,  $[0.35, 0.61]$  oraz  $[0.68, 0.42]$ , gdzie wartości te określają relatywne położenie punktu względem środka układu współrzędnych. Zbiór uczący nie zawiera etykiet klas ani wartości zadanych, ponieważ sieć SOM w sposób nienadzorowany dokonuje aproksymacji topologii rozkładu danych, dążąc do jak najdokładniejszego pokrycia obszaru zdefiniowanego przez ciąg wejściowy. [1]

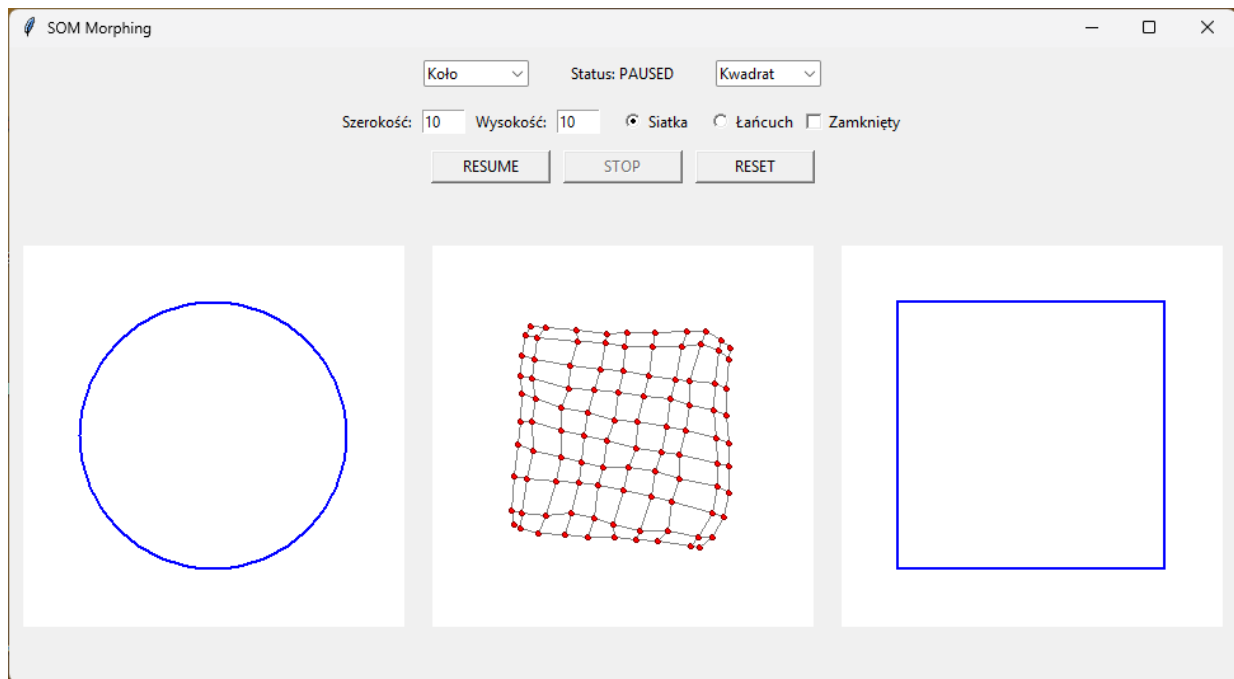
### 4. Ciąg testowy (CT)

Do weryfikacji poprawności działania algorytmu oraz oceny jakości odwzorowania topologii przez sieć, przygotowany został ciąg testowy o liczności 2000 elementów. Zbiór ten został wygenerowany analogicznie do danych uczących, co pozwala rzetelnie sprawdzić, jak sieć radzi sobie z generalizacją kształtu na nowych, nieznanach wcześniej próbkach. Każdy element ciągu to wektor dwuwymiarowy, który określa współrzędne punktu w przestrzeni jednostkowej. W procesie tym dane nie posiadają żadnych etykiet klas - ich jedynym zadaniem jest reprezentowanie rozkładu wybranej figury geometrycznej, co umożliwia porównanie rozłożenia wag neuronów z rzeczywistym kształtem obszaru. Przykładowo, dla trójkąta elementy ciągu mogą przyjmować wartości takie jak  $[0.45, 0.25]$ ,  $[0.20, 0.75]$  oraz  $[0.78, 0.80]$ . Wykorzystanie osobnego zestawu danych testowych pozwala ocenić, czy proces samoorganizacji przebiegł prawidłowo i czy siatka wag równomiernie pokrywa zadany obszar bez tendencji do nadmiernego dopasowania się do pojedynczych punktów ze zbioru uczącego. [1]

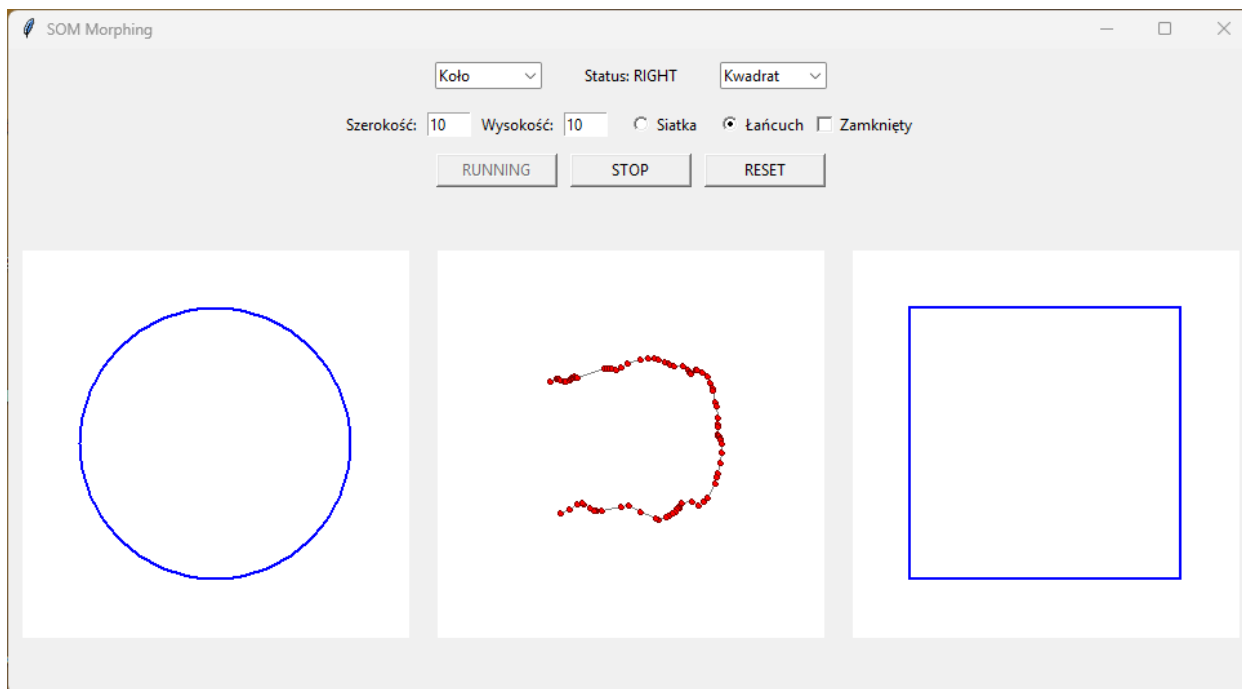
### 5. Implementacja

Aplikacja została zaimplementowana w języku Python z wykorzystaniem biblioteki Tkinter do budowy graficznego interfejsu użytkownika oraz biblioteki NumPy do przeprowadzania wydajnych operacji macierzowych i obliczeń numerycznych związanych z algorytmem SOM. Interfejs programu składa się z panelu sterowania, w którym użytkownik może wybrać kształt źródłowy oraz docelowy (np. koło, kwadrat, trójkąt lub diament), a także określić parametry strukturalne sieci, takie jak szerokość i wysokość siatki neuronów (Rys. 1). Aplikacja oferuje dwa główne tryby wizualizacji: siatkę prostokątną (Grid) oraz łańcuch (Chain), co pozwala na obserwację różnych sposobów mapowania i aproksymacji przestrzeni. Główny silnik obliczeniowy zawarto w klasie *SOM*, która odpowiada za inicjalizację wag oraz

realizację kroku uczenia poprzez wyznaczanie neuronu zwycięzcy (BMU - *Best Matching Unit*) i aktualizację wag w zadanym sąsiedztwie. Kluczowym elementem kodu jest pętla animacji *loop*, która w czasie rzeczywistym aktualizuje widok na trzech kanwasach: lewy i prawy prezentują zbiory punktów (kształty wzorcowe), natomiast środkowy wyświetla dynamicznie deformującą się strukturę sieci neuronowej podczas procesu morfingu. Wizualizacja (Rys. 2) przedstawia proces adaptacji w trybie łańcucha, gdzie czerwone punkty reprezentują wagi neuronów, a szare linie łączą sąsiadujące ze sobą jednostki. Mechanizm „budzenia” parametrów (*wake\_up\_for\_morph*) zapewnia płynne przejście sieci między różnymi kształtami. Gdy współczynnik uczenia  $\eta$  spadnie poniżej zdefiniowanego progu, aplikacja automatycznie resetuje parametry plastyczności sieci, co pozwala jej na skuteczną reorientację i dopasowanie do nowego kształtu docelowego [1].



**Rys.1 Wizualizacja procesu uczenia sieci SOM w trybie siatki**  
**źródło :Opracowanie własne**



*Rys.2. Wizualizacja procesu uczenia sieci SOM w trybie łańcucha  
źródło : Opracowanie własne*

## Instrukcja obsługi aplikacji "SOM"

### 1. Konfiguracja struktury sieci

Przed rozpoczęciem procesu należy ustawić parametry architektury sieci w panelu środkowym:

- **Szerokość i Wysokość:** Pola tekstowe pozwalające zdefiniować liczbę neuronów w siatce (np. 20 X 20 daje 400 neuronów).
- **Siatka / Łańcuch:** Przełączniki typu *Radiobutton* wybierające topologię sieci. Siatka najlepiej wypełnia wnętrza figur, a łańcuch odwzorowuje ich kontury.
- **Zamknięty łańcuch:** Opcja dostępna w trybie łańcucha, pozwalająca połączyć pierwszy neuron z ostatnim, tworząc pętlę (pierścień).

### 2. Wybór danych uczących

Aplikacja posiada dwa niezależne zestawy danych (lewy i prawy):

- **Listy rozwijalne (Combobox):** Pozwalają wybrać kształt docelowy (np. Koło, Trójkąt, Gwiazda) dla lewego i prawego panelu.

### 3. Sterowanie procesem

- **Przycisk START:** Po kliknięciu sieć zaczyna dostosowywać swoje wagi do aktualnie wybranego kształtu.
- **Przycisk STOP:** Zatrzymuje działanie algorytmu w dowolnym momencie jak działa
- **Przycisk RESET:** Przerywa działanie algorytmu, przywraca losowe ustawienie wag neuronów i pozwala na zmianę wymiarów siatki.
- **Morfing w czasie rzeczywistym:** W dowolnym momencie pracy można zmienić kształt docelowy lub przełączyć się między lewym a prawym zestawem danych. Aktywuje to mechanizm „wake up”, dzięki któremu sieć płynnie przepływa z jednej formy w drugą.

#### 4. Interpretacja wizualna

- **Panele boczne:** Wyświetlają obrys figur wybranych jako cel uczenia (kolor niebieski).
- **Panel środkowy:** Prezentuje aktualny stan sieci SOM. Czerwone punkty to neurony, a szare linie to połączenia topologiczne między nimi. Proces kończy się, gdy czerwona struktura równomiernie pokryje zadany kształt.

##### 5.1. Opis najważniejszych fragmentów kodu

Poniższy fragment kodu przedstawia metodę inicjalizującą (`__init__`) klasy SOM, która definiuje architekturę sieci neuronowej. W zależności od wybranego trybu pracy (siatka - *grid* lub łańcuch - *chain*), wagi neuronów są generowane jako macierz trójwymiarowa lub dwuwymiarowa. Wykorzystanie funkcji `np.random.rand` zapewnia losowe rozproszenie wektorów wag w przestrzeni jednostkowej przed rozpoczęciem algorytmu, co jest kluczowe dla prawidłowego przebiegu procesu samoorganizacji. Na końcu metody wywoływana jest funkcja `reset_learning_params`, odpowiedzialna za ustawienie początkowych wartości hiperparametrów sieci: współczynnika szybkości uczenia oraz promienia sąsiedztwa. [5]

```
60 class SOM:
61     def __init__(self, w=10, h=10, mode="grid"):
62         self.mode = mode
63         self.w = w
64         self.h = h
65         self.n = w * h
66         if mode == "grid":
67             self.weights = np.random.rand(self.h, self.w, 2)
68         else:
69             self.weights = np.random.rand(self.n, 2)
70
71         self.reset_learning_params()
72
73     def reset_learning_params(self):
```

Rys.3. Konstruktor klasy SOM i inicjalizacja struktury wag  
źródło :Opracowanie własne

Na rysunku przedstawiono metodę `step`, która realizuje algorytm Kohonena w dwóch dostępnych trybach: dwuwymiarowej siatki (*grid*) oraz jednowymiarowego łańcucha (*chain*). Implementacja obejmuje fazę rywalizacji, polegającą na wyznaczeniu dystansu euklidesowego i lokalizacji neuronu zwycięzcy (BMU - *Best Matching Unit*), oraz fazę adaptacji, w której wagi neuronów znajdujących się w promieniu sąsiedztwa  $S$  ulegają modyfikacji. W liniach 104-105 widoczny jest mechanizm wykładniczego wygaszania parametrów: współczynnika szybkości uczenia  $\eta$  oraz promienia  $S$  przy użyciu stałego mnożnika 0.9998. [5]

```

86
87     def step(self, p):
88         if self.mode == "grid":
89             d = np.sum((self.weights - p)**2, axis=2)
90             i, j = np.unravel_index(np.argmin(d), d.shape)
91             for y in range(self.h):
92                 for x in range(self.w):
93                     dist = np.sqrt((i-y)**2 + (j-x)**2)
94                     if dist < self.S:
95                         self.weights[y,x] += self.eta * (p - self.weights[y,x])
96         else:
97             d = np.sum((self.weights - p)**2, axis=1)
98             idx = np.argmin(d)
99             for i in range(len(self.weights)):
100                 dist = abs(i - idx)
101                 if dist < self.S:
102                     self.weights[i] += self.eta * (p - self.weights[i])
103
104         self.eta *= 0.9998
105         self.S *= 0.9998
106

```

**Rys.4. Implementacja kluczowej procedury uczenia sieci SOM**  
**źródło :Opracowanie własne**

Na poniższym rysunku przedstawiono fragment kodu odpowiedzialny za dynamiczną kontrolę plastyczności sieci SOM. Metoda `reset_learning_params` definiuje wejściowe wartości hiperparametrów dla nowej sesji uczenia, ustalając bazowy poziom współczynnika szybkości uczenia ( $\eta$ ) oraz promienia sąsiedztwa ( $S$ ). Z kolei metoda `wake_up_for_morph` (linie 80-85) implementuje specyficzną funkcjonalność przywracania wyższych wartości parametrów  $\eta$  i  $S$  w momencie zmiany kształtu docelowego danych. Pozwala to na skuteczne przerwanie stanu stabilizacji sieci i umożliwia jej ponowną, płynną reorganizację (morfing) do nowej geometrii, eliminując konieczność całkowitego resetowania wag neuronów. [5]

```

73     def reset_learning_params(self):
74         self.eta = 0.1
75         if self.mode == "grid":
76             self.S = max(self.w, self.h) / 2
77         else:
78             self.S = self.n / 2
79
80     def wake_up_for_morph(self):
81         self.eta = 0.05
82         if self.mode == "grid":
83             self.S = max(self.w, self.h) / 4
84         else:
85             self.S = self.n / 4
86

```

**Rys.5. Metody zarządzania parametrami uczenia i mechanizm „wake up”**  
**źródło :Opracowanie własne**

Na poniższym rysunku przedstawiono przykładową implementację funkcji `circle_pts`, odpowiedzialnej za dynamiczne generowanie zbioru danych w kształcie koła. Algorytm wykorzystuje współrzędne biegunowe, w których promień  $r$  oraz kąt  $a$  są wyznaczone na podstawie rozkładu jednostajnego. Zastosowanie pierwiastka kwadratowego przy losowaniu promienia (linia 9) jest kluczowe dla uzyskania homogenicznego rozkładu punktów, gdyż zapobiega to ich nienaturalnemu zagęszczeniu w centrum figury. Finalnie, funkcja `np.column_stack` (linia 11) dokonuje transformacji współrzędnych do układu kartezjańskiego  $[x, y]$ , tworząc wektory wejściowe gotowe do przetworzenia przez sieć SOM. [5]

```

8  def circle_pts(n):
9      r = np.sqrt(np.random.rand(n)) * 0.35
10     a = np.random.rand(n) * 2*np.pi
11     return np.column_stack((0.5 + r*np.cos(a), 0.5 + r*np.sin(a)))
12

```

**Rys.6. Implementacja funkcji generującej dwuwymiarowy ciąg uczący**  
**źródło :Opracowanie własne**

## 6. Proces uczenia

Adaptacja sieci odbywa się w drodze iteracyjnego procesu uczenia nienadzorowanego, opartego na regule Kohonena. Każdy krok algorytmu rozpoczyna się od pobrania z ciągu uczącego losowego punktu, który reprezentuje współrzędne w przestrzeni dwuwymiarowej. Następnie sieć przeprowadza fazę rywalizacji, w której sprawdzane są wagi wszystkich neuronów w celu wyłonienia jednostki najlepiej dopasowanej, czyli tzw. zwycięzcy. Po wyznaczeniu zwycięskiego neuronu następuje faza aktualizacji wag. Modyfikacji podlegają wektory wag zwycięzcy oraz wszystkich neuronów znajdujących się w jego bezpośrednim sąsiedztwie topologicznym. Zasięg tych zmian określa promień sąsiedztwa, a ich intensywność współczynnik szybkości uczenia. Kluczowym elementem zapewniającym stabilność algorytmu jest systematyczne zmniejszanie wartości obu tych parametrów wraz z upływem czasu. Dzięki temu sieć początkowo wykonuje gwałtowne ruchy, szybko rozciągając się na cały obszar danych, a w późniejszych etapach jedynie precyzyjnie koryguje swoje położenie. Dodatkowo w programie zaimplementowano funkcję odświeżania parametrów (tzw. "wake up"). W momencie zmiany kształtu docelowego przez użytkownika, wartości szybkości uczenia i promienia sąsiedztwa są skokowo podnoszone. Przywraca to sieci elastyczność i pozwala na płynną migrację neuronów do nowej formy geometrycznej, co w interfejsie graficznym

widoczne jest jako efektowny proces morfingu struktury.[5]

## 7. Testowanie

Testowanie sieci polegało głównie na sprawdzeniu, jak mapa radzi sobie z odwzorowaniem różnych figur geometrycznych oraz jak przebiega proces morfingu przy nagłej zmianie kształtu docelowego. W trakcie prób sieć była "rzucana" na różne zestawy danych - od prostego kwadratu po skomplikowaną gwiazdę. Skuteczność oceniałem wizualnie, obserwując, czy czerwone punkty (neurony) równomiernie pokrywają niebieski obrys figury i czy połączenia między nimi nie tworzą dziwnych supłów (tzw. martwych neuronów). W przypadku trybu siatki, sieć bez większych problemów wypełniała całą powierzchnię figur, choć przy gwiazdach o ostrych ramionach potrzebowała nieco więcej czasu, żeby wagi "dociągnęły" do samych końców. Z kolei tryb łańcucha bardzo ładnie układał się wzdłuż krawędzi, tworząc czytelny obrys kształtu. Uzyskana skuteczność jest w pełni zadowalająca - po kilkunastu sekundach (kilkudziesięciu epokach) sieć stabilizowała się, a błąd odwzorowania stawał się niemal niewidoczny gołym okiem. Najciekawiej testy wypadły przy przełączaniu celu w trakcie pracy; dzięki funkcji sieć sprawnie porzucała starą strukturę i płynnie "przepływała" w nowy kształt, co potwierdziło, że zaimplementowany mechanizm adaptacji działa bez zarzutu. [5]

## 8. Wnioski

Praca z modelem samoorganizujących się map Kohonena potwierdziła niezwykłą zdolność sieci neuronowych do nienadzorowanej ekstrakcji cech oraz autonomicznego dopasowywania się do struktury danych bez użycia zewnętrznych etykiet. Najważniejszym technologicznym spostrzeżeniem jest kluczowa rola mechanizmu wygaszania parametrów, ponieważ utrzymywanie współczynnika uczenia na stałym, wysokim poziomie prowadzi do permanentnej niestabilności i gwałtownych oscylacji wag. W procesie adaptacji do zmieniającej się geometrii niezbędny okazał się mechanizm „wake up”, który przywraca sieci elastyczność i umożliwia płynny morfing wag do nowych kształtów. Eksperymenty wykazały również istotne różnice w zastosowaniu konkretnych topologii: podczas gdy siatka najlepiej wypełnia pełne obszary dwuwymiarowe, tryb łańcucha wykazuje znacznie wyższą skuteczność w aproksymacji samych konturów. Należy jednak pamiętać o tempie redukcji promienia sąsiedztwa, gdyż zbyt pośpieszne ograniczanie tego parametru skutkuje powstawaniem błędów topologicznych, czyli tzw. supłów, które blokują sieć przed osiągnięciem pożądanej formy. Ostatecznie, mimo unikalnego przebiegu każdej sesji wynikającego z losowości danych, systematyczne dążenie sieci do poprawnego odwzorowania geometrii oraz stabilne działanie aplikacji z wizualizacją czasu rzeczywistego potwierdzają wysoką efektywność tej metody w analizie struktur danych.

## 9. Bibliografia

- [1] Dokumentacja biblioteki NumPy, *NumPy v1.26 Manual*, <https://numpy.org/doc/> (dostęp: 2024).
- [2] Dokumentacja biblioteki Tkinter, *Python interface to Tcl/Tk*, <https://docs.python.org/3/library/tkinter.html> (dostęp: 2024).
- [3] Kohonen T., *Self-Organizing Maps*, Springer-Verlag, Berlin Heidelberg 2001.
- [5] Wikipedia, wolna encyklopedia, *Samoorganizująca się mapa (Hasło: Sieć Kohonena)*, [https://pl.wikipedia.org/wiki/Samoorganizuj%C4%85ca\\_si%C4%99\\_mapa](https://pl.wikipedia.org/wiki/Samoorganizuj%C4%85ca_si%C4%99_mapa) (dostęp: 2024).

| WKŁAD DO PROJEKTU                                               |                                        |                                                      |
|-----------------------------------------------------------------|----------------------------------------|------------------------------------------------------|
| Ivan Pohodaev                                                   | Andrii Yevlakh                         | Maksymilia Wieczorek                                 |
| Poprawki kodu (napisanie trzeciej wersji )<br>edycja sprawoznie | Pisanie kodu (napisanie dwóch wersji ) | Stworzeie sprawozdania i edycja , małe poprawki kodu |